

Academia Perruna

Queremos modelar el comportamiento de los animales que fueron enrolados a una academia de entrenamiento perruno.

De cada perro nos interesa registrar su nombre, la cantidad de estamina que tiene, cuál es su juguete y su criterio de juego, que determina cuándo el animal en cuestión se siente a gusto con otro perro para poder jugar con él. De momento, de los juguetes nos indican el color y si rechinan o no cuando los muerden, aunque seguramente aparezcan otras características relevantes en el futuro.



Resolver los siguientes requerimientos aprovechando los conceptos del paradigma funcional, favoreciendo cuando sea conveniente el uso de **orden superior**, **aplicación parcial** y **composición** a lo largo del desarrollo.

Para todas las funciones desarrolladas, **explicitar su tipo**.

1.

- a. Necesitamos modelar a los perros y juguetes de acuerdo a lo indicado previamente y tener un ejemplo de un perro que se llame Sultán, con 25 unidades de estamina, un juguete azul que no rechina y que le guste jugar con otro perro si el juguete que tiene sí rechina.

Se cuenta con las siguientes definiciones:

<pre>data Perro = Perro { nombre :: String, estamina :: Number, juguete :: Juguete, juego :: Bool } type Juguete = (String, Bool)</pre>	<pre>sultan = Perro { nombre = "Sultán", estamina = 25, juguete = ("azul", False), juego = rechina (juguete otroPerro) } rechina = snd</pre>
---	--

Esta solución tiene problema/s. Explicar qué problemas se identificaron y corregirlos.

- b. Se necesita una función `recibirJuguete :: Juguete -> Perro -> Perro`, sabiendo que cuando esto sucede, además de quedarse con el juguete nuevo, su estamina sube 3 puntos por el entusiasmo.

Ya se cuenta con estas dos funciones:

```
jugueteNuevo perro nuevo = perro {juguete = nuevo}  
cambiarEstamina x perro = perro {estamina = x (estamina perro)}
```

Definir recibirJuguete utilizando ambas funciones.

- c. Hacer una función `puedenJugarJuntos :: Perro -> Perro -> Bool` que se cumpla si mutuamente cumplen el criterio de juego del otro.
- d. Hacer una función que vuelva más exigente a un perro para poder jugar con otro. Dado un nuevo criterio de juego, se espera que el perro luego de volverse más exigente, requiera que el otro perro cumpla no sólo su criterio previo, sino también el nuevo criterio indicado.
Mostrar un ejemplo de uso haciendo que Sultán ahora también requiera que el otro perro tenga más de 10 de estamina para que pueda jugar con él.

2. En la academia les arman rutinas para hacer a los perritos durante el día, por lo que les interesa poder analizar el estado final de un perro luego de realizar sucesivamente múltiples acciones. Sabiendo que una Tarea es una función de tipo `Perro -> Perro`, y asumiendo que ya existen las siguientes funciones (no importa cómo están implementadas):

<code>comer :: Tarea</code>	<code>correr :: Tiempo -> Tarea</code>	<code>saltar :: Tarea</code>
-----------------------------	---	------------------------------

- a. Definir una función que, dada una lista de tareas que representa la rutina a realizar y un perro, retorne cómo quedaría el perro en cuestión si realizara cada una de esas tareas sucesivamente.
 - b. Saber si una rutina es muy exigente para un perro, que se cumple si luego de realizar las tareas gastaría más del 50% de su estamina original.
 - c. Mostrar un ejemplo de uso de la función anterior que permita saber si una rutina que haga que Sultán coma, luego corra 10 minutos y por último salte sería muy exigente.
3. Queremos que los perros puedan elegir un nuevo juguete de entre aquellos que hay disponibles como una nueva tarea que pueden realizar. Usando la función `recibirJuguete`, definir `elegirNuevoJuguete` que reciba una lista de juguetes y una función de tipo `Juguete -> Juguete -> Bool` que sirva para saber si el segundo juguete es mejor que el primero. Lo esperado para esta tarea es que el perro reciba un nuevo juguete, cambiando el propio por el primero de los juguetes dados que cumpla con ser mejor que el juguete que tiene actualmente. En caso de no haber ningún juguete disponible que sea mejor que el que ya tiene, no pasa nada, debería conservar el propio.
4. Teniendo una constante `perrosDeAcademia :: [Perro]`, queremos obtener los nombres de los 5 primeros perros que podrían jugar con Sultán.
- a. Sin definir nuevas funciones, mostrar cuál sería la consulta que debería hacerse para lograr lo pedido usando **composición de funciones**.
 - b. Si `perrosDeAcademia` fuera una lista infinita, ¿sería posible obtener una respuesta para la consulta anterior? Justificar.